

INSTITUTO TECNOLÓGICO DE COSTA RICA
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación

IC3101 – Arquitectura de Computadoras
II Semestre 2024

Lectura #1 - Semana 2 – Capítulo 4
La arquitectura IA-32

Estudiante: Mauricio González Prendas – **Carné:** 2024143009

Profesor: Esteban Arias Méndez

Fecha de entrega: 05/08/2024

Bibliografía:

[1] S. P. Dandamudi, *Guide to Assembly Language Programming in Linux*.
Springer Science & Business Media, 2005, pp. 61–78.

Abstract—*The processor's execution cycle involves fetching, decoding, and executing instructions, using registers to handle data and control operations. In protected mode, the processor uses 32-bit addresses with techniques like segmentation and paging, while in real mode, it utilizes 20-bit addresses and 64 KB segments. Mixed mode allows combining operands and addresses of different sizes. I/O devices connect to the system through controllers and ports, with access methods including memory-mapped I/O and isolated I/O.*

Ciclo de ejecución del procesador

1. **Fetch (Obtener):** El procesador obtiene una instrucción de la memoria.
2. **Decode (Decodificar):** El procesador identifica la instrucción obtenida. Las instrucciones en lenguaje máquina siguen un esquema de codificación específico que facilita este proceso.
3. **Execute (Ejecutar):** La ALU realiza operaciones aritméticas y lógicas sobre los datos, mientras que los circuitos de control proporcionan el control de tiempo y las instrucciones para llevar a cabo la operación.

En la práctica, las instrucciones y datos suelen no ser obtenidos directamente de la memoria principal, sino que se accede a través de una memoria caché de alta velocidad que proporciona un acceso más rápido.

Registros de Datos: Se utilizan en la mayoría de las instrucciones aritméticas y lógicas, aunque algunos tienen funciones especiales para instrucciones específicas.

Registros de índice: Tienen un papel especial en las instrucciones de procesamiento de cadenas.

Fetch	Decode	Execute	Fetch	Decode	Execute	Fetch ...
-------	--------	---------	-------	--------	---------	-----------

The diagram illustrates the structure of 32-bit registers in x86 assembly. A horizontal bar at the top indicates a total width of 32 bits, divided into a 16-bit segment and two 8-bit segments. Below this, a table lists the registers and their sub-registers:

Register	16-bit Sub-register	8-bit Sub-register 1	8-bit Sub-register 2	Description
EAX	AX	AH	AL	AX = Accumulator / Acumulador
EBX	BX	BH	BL	BX = Base
ECX	CX	CH	CL	CX = Counter / Contador
EDX	DX	DH	DL	DX = Data
ESI	SI		SIL	Source index
EDI	DI		DIL	Destination index
ESP	SP		SPL	Stack pointer
EBP	BP		BPL	Base pointer

At the bottom, a horizontal bar indicates the full 32-bit width of the registers.

s relacionadas con interrupciones.

FLAGS

3 1		2 2	2 10	2 9	1 8	1 7	1 6	1 5	1 4	1 3	1 2	1 1	1 0	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	I D	V P	V F	A C	M F	R F	N T	O PL	O FF	D I	T S	Z O	F A	P F	C F

EFLAGS

ID = ID flag

Registros de Segmento

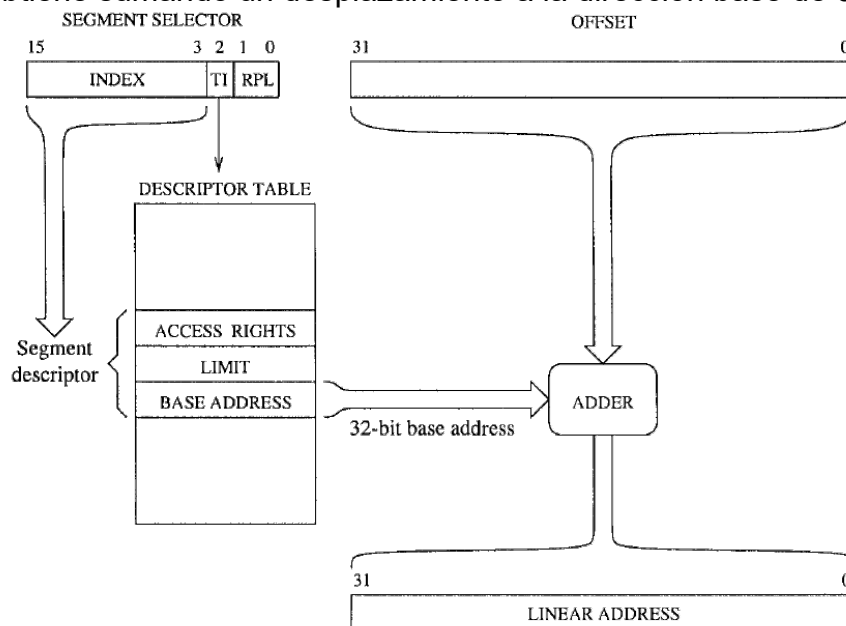
En un programa, hay una división lógica entre las instrucciones y los datos. El registro de segmento de código (CS) señala la ubicación de las instrucciones del programa en la memoria, mientras que el registro de segmento de datos (DS) apunta a la parte del programa que contiene los datos. Además, el registro de segmento de pila (SS) se encarga de indicar la ubicación del segmento de pila del programa, el cual se explica con más detalle en capítulos posteriores.

Los otros tres registros de segmento, ES, FS, y GS, son adicionales y se pueden utilizar de manera similar a los registros de segmento principales.

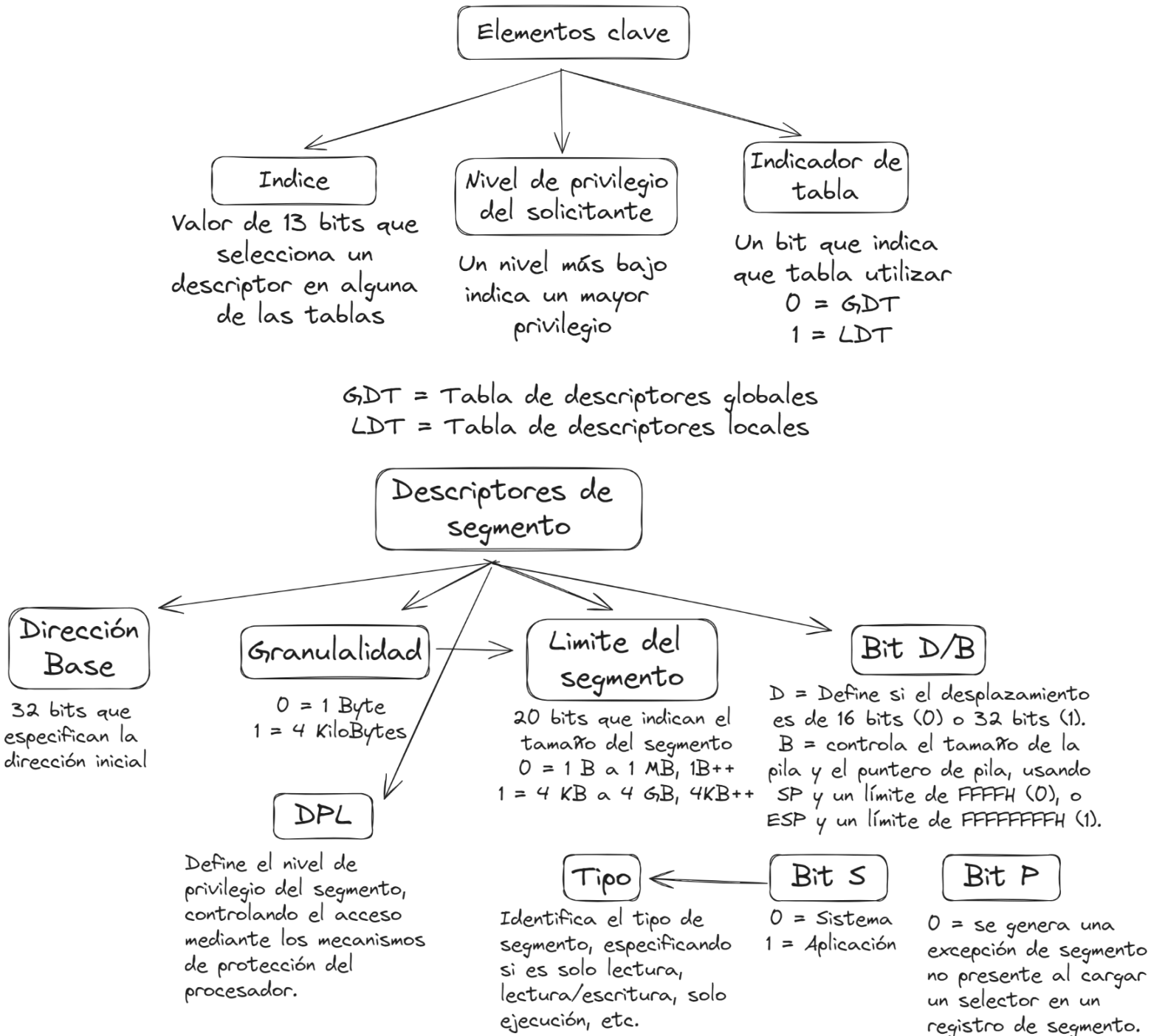
15	0
CS	Code segment
DS	Data segment
SS	Stack segment
ES	Extra segment
FS	Extra segment
GS	Extra segment

Arquitectura de memoria en modo protegido

La arquitectura IA-32 soporta tanto el modo real como el modo protegido, siendo este último su modo nativo. En el modo protegido, se utilizan direcciones de 32 bits y se admite tanto segmentación como paginación. En este modo, una dirección lógica se traduce a una dirección lineal de 32 bits mediante segmentación, y luego, si se emplea paginación, se convierte en una dirección física. Los registros de segmento actúan como índices en una tabla de descriptores que contienen la dirección base, el tamaño y los derechos de acceso del segmento. La dirección lineal se obtiene sumando un desplazamiento a la dirección base de 32 bits.



Cada registro de segmento tiene una parte "visible" y una "invisible". La parte visible, conocida como el selector de segmento, es un valor de 16 bits que puede ser cargado directamente mediante instrucciones como mov, pop, lds, les, lss, lgs, y lfs. La parte invisible es gestionada automáticamente por el procesador desde una tabla de descriptores.



DPL = Nivel de privilegio del descriptor

GDT = Tabla de descriptores globales
LDT = Tabla de descriptores locales
IDT = Tabla de descriptores de interrupciones

Tablas de descriptores de segmento

Arreglo de descriptores de segmento

GDT

LDT

IDT

Contiene descriptores accesibles por todas las tareas del sistema. Solo hay una, que típicamente incluye el código y los datos del sistema operativo. Puede tener hasta 8192 descriptores y se maneja mediante el registro GDTR. Las instrucciones lgdt y sgdt se utilizan para cargar y almacenar la GDT.

Contiene descriptores específicos para programas individuales. Puede haber varias, cada una con descriptores para código, datos, pila, etc. Puede tener hasta 8192 descriptores y se maneja mediante el registro LDTR. Las instrucciones lldt y sltd se utilizan para cargar y almacenar la LDT.

Gestiona el procesamiento de interrupciones y puede tener un tamaño variable entre 8 bytes y 64 KB. Es gestionada implícitamente por el registro IDTR.

Modelos de segmentación

Sijat

Multisegmento

Mapea todas las direcciones base de segmento a cero y establece el tamaño a 4 GB, haciendo que la segmentación sea invisible.

Se utiliza en entornos UNIX y Linux

Permite la definición de múltiples segmentos en un programa.

Solo los segmentos activos están cargados en los registros de segmento. Los segmentos inactivos se activan cargando su selector en un registro. Si se accede a memoria fuera del límite del segmento, el procesador genera una excepción de protección general.

Arquitectura de memoria en modo real

Debido a que todos los registros del 8086 son de 16 bits, el espacio de direcciones está limitado a 64 KB, lo que lleva a organizar la memoria en segmentos de hasta 64 KB. Para identificar una ubicación en la memoria, se utilizan dos componentes: la base del segmento, que especifica el inicio del segmento en la memoria, y el desplazamiento, que indica la dirección relativa dentro del segmento. Esta combinación se denomina dirección lógica.

Aunque los programadores trabajan con direcciones lógicas, el procesador utiliza direcciones físicas de 20 bits. La conversión de una dirección lógica a física se realiza añadiendo 4 bits menos significativos de 0 a la base del segmento y luego sumando el desplazamiento. Por ejemplo, para calcular la dirección física de la dirección lógica 1100:450H, se añaden los 4 bits menos significativos a la base del segmento y se suma el desplazamiento correspondiente.

$$\begin{array}{r} 11000 \\ + \quad 450 \\ \hline = 11450 \end{array}$$

Esto da como resultado la dirección física: 11450.

* Cada dirección lógica tiene una dirección física única, pero una dirección física puede corresponder a más de una dirección lógica.

En modo real, el tamaño de un segmento está limitado a 64 KB debido al desplazamiento de 16 bits. El procesador define cada segmento en exactamente 64 KB. Un programa puede acceder a hasta seis segmentos simultáneamente. Aunque el 8086 soportaba solo cuatro segmentos (sin registros FS y GS), en ensamblador se usan típicamente al menos dos segmentos: código y pila, y un tercer segmento para datos si el programa los requiere. Los segmentos pueden ubicarse en cualquier lugar de la memoria siempre que comiencen en límites de 16 bytes, y los registros de segmento son independientes, permitiendo que los segmentos sean contiguos, disjuntos, parcialmente solapados o completamente solapados.

Operación en Modo Mezclado

En los modos protegido y real, se pueden usar operandos y direcciones tanto de 16 bits como de 32 bits. El bit D/B determina el tamaño predeterminado. Para permitir el uso combinado de tamaños, el conjunto de instrucciones ofrece dos prefijos de anulación de tamaño: uno para operandos y otro para direcciones. Estos prefijos permiten la programación en modo mezclado, facilitando la combinación de operandos y direcciones de diferentes tamaños en las instrucciones.

Entrada/Salida (I/O)

Los dispositivos I/O permiten a un sistema informático interactuar con el mundo exterior. Pueden ser de entrada (teclado, ratón), de salida (impresora, pantalla) o

ambos (disco). Aunque se comunican a través del bus del sistema, lo hacen mediante un controlador que actúa como intermediario.

Razones para usar un controlador:

1. **Características de dispositivos:** Cada dispositivo tiene características distintas. Sin un controlador, el procesador tendría que gestionar cada dispositivo, reduciendo el tiempo para ejecutar programas. El controlador maneja los comandos y datos del dispositivo, liberando al procesador para otras tareas.
2. **Potencia eléctrica:** Los controladores contienen hardware adicional para enviar corriente a través de cables más largos.

Registros internos del controlador I/O:

- **Registro de datos:** Almacena la información que se transferirá.
- **Registro de comandos:** Indica la operación solicitada al controlador.
- **Registro de estado:** Proporciona información sobre el estado del dispositivo (por ejemplo, si está en línea o fuera de papel).

Ejemplo de Operación de I/O: Para imprimir un carácter, el procesador:

1. Verifica el registro de estado para asegurarse de que la impresora esté lista.
2. Coloca el carácter en el registro de datos.
3. Configura el registro de comandos para iniciar la transferencia.

Acceso a Dispositivos I/O:

- **Puertos I/O:** Son direcciones de registros asociadas con un controlador.
- **Mapeo de Puertos I/O:**
 - **I/O mapeado en memoria:** Algunos procesadores asignan puertos a direcciones de memoria.
 - **I/O aislado:** Otros procesadores usan un espacio de direcciones separado para I/O y requieren instrucciones especiales como in y out.

Espacio de Direcciones I/O en IA-32: La arquitectura IA-32 proporciona un espacio de direcciones I/O de 64 KB, que puede ser utilizado para puertos de 8 bits, 16 bits o 32 bits. La combinación total de puertos no debe exceder este espacio.

Control de Dispositivos I/O: Aunque los programadores pueden acceder directamente a los dispositivos I/O mediante lenguaje ensamblador, los sistemas operativos proporcionan rutinas estandarizadas para facilitar el acceso y evitar problemas como el mal uso de recursos. Linux ofrece llamadas al sistema para el acceso a dispositivos I/O, mientras que Windows utiliza el BIOS y el sistema operativo para controlar estos dispositivos a través de interrupciones.